Practica_05 “Manejo de escenas”

Alumno: **Villafaña Pasten Elliot Kenneth 118006642**

Enlace a github: https://github.com/Piabonnie-Dev/practica02-motor.git

PROCESO DE IMPLEMENTACION Y DEPURACION:

Para realizar la Práctica 05 se partió del proyecto desarrollado en la Práctica 04, el cual ya contaba con un sistema de componentes, movimientos del jugador, detección de colisiones AABB, sistema de depuración visual de colliders y un obstaculo con colisiones.

El objetivo principal de esta práctica fue reorganizar el motor de para incorporar un sistema de escenas utilizando una pila de estados. Para ello se agregaron las clases `Scene, SceneManager, TitleScnee, GameScene, PauseScene, y GameOverScene.` Lo que se hizo fue sacar de main.cpp la logica del juego y trasladarla principalmente a GameScene, dejando al archivo principal encargado únicamente de inicializar SDL, calcular el deltaTime, procesar eventos y delegar la ejecución al administrador de escenas.

Durante este proceso surgieron diversos errores de compilación y de lógica. Estos errores fueron corregidos gradualmente, evitando reemplazar completamente el código ya construido para conservar la estructura y estilo de las prácticas anteriores.

1. INTEGRACION INCORRECTA DEL SISTEMA DE ESCENAS

Uno de los principales problemas apareció al implementar `SceneManager.` La función `ChangeScene()` llegó utilizar la acción Push, lo cual provocaba que una nueva escena se agregara encima de la anterior en lugar de sustituirla. Esto podía ocasionar que `TitleScene`, `GameScene` y `GameOverScene` permanecieran acumuladas en memoria.

La diferencia entre ambas operaciones resultó fundamental: `ChangeScene` debe eliminar la escena actual y colocar una nueva, mientras que `PushScene` debe conservar la escena anterior para casos como la pausa.

Este error me permitió comprender mejor la definición de una pila de escenas y la diferencia entre cambiar y superponer una escena.

2. SEPARACION INCORRECTA ENTRE UPDATE Y RENDER

Otro problema ocurrió cuando la clase `Scene.` Durante la implementación se mezclo la logica del renderizado con el metodo `Update`.

Provocando que las escenas derivadas no reconocieran correctamente la función Render(), generando errores en `TitleScene, GameScene, PauseScene y GameOverScene`.

La solución consistio en separar correctamente ambas responsabilidades: `Update(float dt)` quedó encargado de actualizar la lógica de las entidades, mientras que `Render(SDL_Renderer*)` quedo exclusivamente de dibujarlas. La estructura final de Scene mantiene ambas opreaciones independientes.

3. PERDIDA TEMPORAL DEL FUNCIONAMIENTO DE LAS COLISIONES

El problema mas horrible que me toco, fue cuando traslade el código de la Práctica 04 desde main.cpp hacia `GameScene`.

Despues de realizar la refactorización. El programa se veía bien al compilar, pero las colisiones dejaron temporalmente de comportarse de la misma manera que antes.

Se comprobo que el jugador y pelota conservaban sus respectivos colliders, despues de actualizar las entidades.

También se verifico que `BallComponent` conservara su respuesta al evento de colisión para modificar la velocidad y generar el rebote.

Finalmente se logro recuperar el funcionamiento correcto de las colisiones dentro de la nueva arquitectura.

4. MIGRACION DE LA LOGICA DE MAIN.CPP HACIA GAMESCENE

Otro desafío importante fue retirar de `main.cpp` la administración directa de la pelota, los obstáculos, las físicas, colisiones y la del jugador.

En anteriores prácticas todas esas responsabilidades estaban en el estado principal de la aplicación. Pero en esta práctica fue necesario trasladarlas hacia `GameScene`, dejando a main.cpp únicamente con la inicialización de SDL, también con el cálculo `deltaTime`, la delegación de eventos, actualización y renderizado hacia `SceneManager`. Ya la versión final solo mantiene un `SceneManager`.

5. CONFIGURACION ANTERIOR DE CMAKE

También se presentó un problema con la configuración de compilación. Donde `CMake` continuaba utilizando información de la Práctica 04 almacenada en la carpeta `Build`.

Fue necesario eliminar la configuración anterior y regenerarla:

`rm -rf build`

`cmake -B build`

`cmake –build build`

Despues de esto, el proyecto comenzó a compilar correctamente con practica05_motor.exe

6. RESULTADOS OBTENIDOS

Después de corregir los problemas anteriores, se logró compilar correctamente la Práctica 05 y ejecutar el programa utilizando la nueva arquitectura de escenas.

El motor dejó de depender de `main.cpp` para administrar directamente toda la lógica del juego. Ahora cada escena tiene una responsabilidad diferente y `SceneManager` se encarga de controlar cuál se encuentra activa y qué tipo de transición se debe realizar.

La estructura final quedó compuesta principalmente por:

- `TitleScene`: representa la pantalla inicial del programa.
- `GameScene`: contiene la lógica principal del juego, incluyendo al jugador, la pelota, el obstáculo, las físicas y las colisiones.
- `PauseScene`: permite pausar la ejecución del juego sin destruir el estado de `GameScene`.
- `GameOverScene`: representa el estado de finalización de la partida.
- `SceneManager`: administra la pila y los cambios entre las diferentes escenas.

Otro resultado importante fue lograr conservar las funciones realizadas en la Práctica 04. Después de los problemas que surgieron durante la migración, las colisiones AABB volvieron a funcionar correctamente dentro de `GameScene`, junto con el movimiento de la pelota y la visualización de los colliders.

Finalmente, el ejecutable quedó generado correctamente como:

```
practica05_motor.exe
```

7. PRUEBAS DE FUNCIONAMIENTO

Una vez que el proyecto logró compilar correctamente, se realizaron pruebas funcionales para comprobar que el cambio de arquitectura no solamente funcionara a nivel de código, sino también durante la ejecución.

La primera prueba consistió en iniciar el programa y comprobar que apareciera la escena inicial antes de comenzar el juego.

Posteriormente se ingresó a `GameScene` para revisar que los elementos desarrollados anteriormente continuaran funcionando. Se verificó principalmente el movimiento del jugador, el movimiento de la pelota, las colisiones y el comportamiento del obstáculo.

También fue importante volver a revisar las colisiones después del problema que ocurrió durante la migración desde `main.cpp`, ya que durante una etapa del desarrollo la pelota dejó de reaccionar correctamente con el jugador. Después de corregir el código, la detección y el rebote volvieron a funcionar.

Para comprobar visualmente los colliders se utilizó nuevamente el sistema de depuración mediante F1.

También se probaron las transiciones principales del sistema de escenas:

```
TitleScene
     ↓
GameScene
     ↓
PauseScene
     ↓
GameScene
     ↓
GameOverScene
```

La pausa debía conservar el estado del juego y, al salir de ella, el programa debía continuar desde el punto en el que se encontraba anteriormente.

Por otra parte, al cambiar completamente de una escena a otra se comprobó que la escena anterior fuera sustituida y no se acumulara innecesariamente dentro de la pila.

8. PRUEBA DE COMPILACION LIMPIA

Como prueba final también se realizó una compilación desde cero para comprobar que el proyecto no dependiera de archivos temporales generados anteriormente.

Para esto se eliminó nuevamente la carpeta `build`:

```
rm -rf build
```

Después se volvió a configurar el proyecto:

```
cmake -B build
```

Y finalmente se compiló:

```
cmake --build build
```

Esta prueba fue importante porque anteriormente CMake todavía conservaba configuraciones de la Práctica 04.

Al realizar una compilación limpia se pudo comprobar que la configuración actual corresponde realmente a la Práctica 05 y que todos los archivos necesarios se encuentran correctamente incluidos dentro del proyecto.

9. APRENDIZAJES OBTENIDOS

Esta práctica permitió comprender mejor cómo se puede organizar un motor cuando comienza a aumentar la cantidad de estados y funcionalidades.

En las prácticas anteriores gran parte de la lógica estaba concentrada directamente en `main.cpp`, lo cual era suficiente cuando el programa todavía era pequeño. Sin embargo, al agregar un menú, una pausa y una pantalla de Game Over, mantener toda esa lógica dentro del archivo principal comenzaba a ser poco práctico.

El uso de una clase base `Scene` permitió que diferentes estados del programa compartieran una misma estructura mediante métodos como:

```
Init()
Exit()
HandleEvent()
Update()
Render()
```

También se comprendió mejor la diferencia entre `ChangeScene`, `PushScene` y `PopScene`.

Antes de esta práctica no resultaba tan evidente por qué una pausa no debía implementarse simplemente cambiando de escena. Al utilizar una pila se pudo observar que `PauseScene` puede colocarse encima de `GameScene`, detener

temporalmente su actualización y posteriormente eliminarse para regresar exactamente al mismo estado del juego.

Otro aprendizaje importante estuvo relacionado con las colisiones. Cuando se trasladó la lógica del juego hacia `GameScene`, algunas partes necesarias para mantener funcionando el sistema de colisiones dejaron de ejecutarse de la forma esperada. Esto demostró que durante una refactorización no basta con verificar que el código compile, sino que también es necesario volver a probar las funcionalidades que ya existían.

10. CONCLUSION

En esta práctica se logró implementar un sistema de manejo de escenas para reorganizar el motor desarrollado durante las prácticas anteriores.

El principal cambio consistió en separar los diferentes estados del programa en clases independientes, evitando que toda la responsabilidad permaneciera concentrada dentro de `main.cpp`.

Durante la implementación surgieron diversos problemas, principalmente relacionados con el funcionamiento de la pila de escenas, la separación entre `Update` y `Render`, la migración de la lógica del juego hacia `GameScene`, la configuración de CMake y la pérdida temporal del comportamiento correcto de las colisiones.

El problema de las colisiones fue uno de los más importantes porque el programa podía compilar y mostrarse correctamente en pantalla, pero una función que ya trabajaba en la Práctica 04 había dejado de comportarse de la misma manera. Esto permitió comprender la importancia de realizar pruebas después de modificar la arquitectura de un programa.

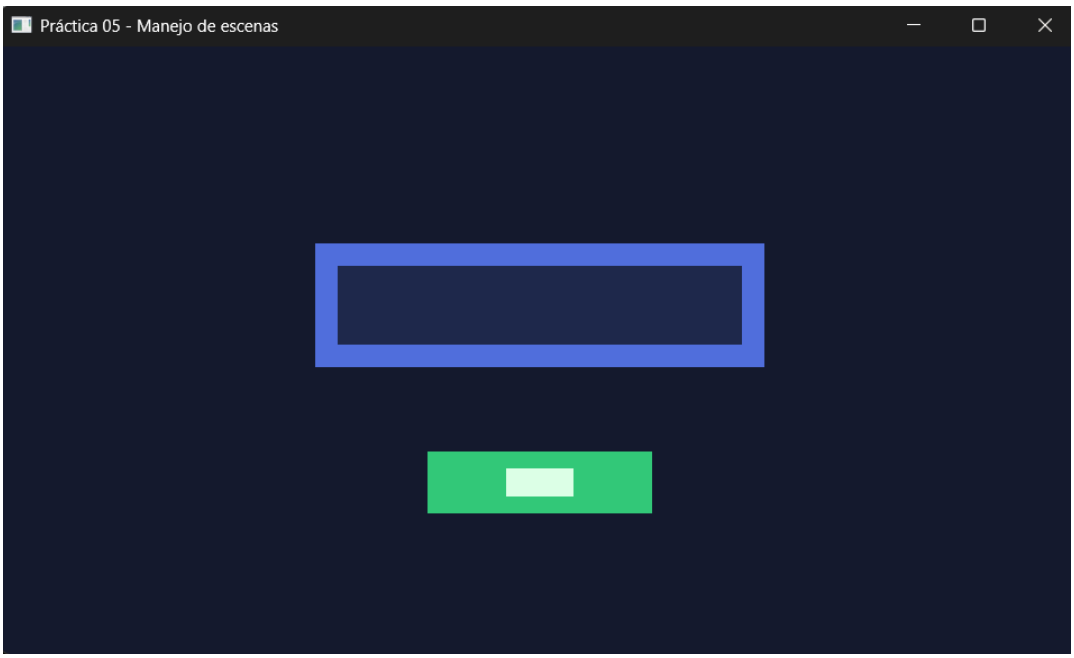
Al final se consiguió integrar el nuevo sistema de escenas sin perder las principales funcionalidades desarrolladas anteriormente.

La práctica permitió que el proyecto dejara de funcionar únicamente como una demostración de componentes individuales y comenzara a tener una estructura más cercana a la que puede utilizar un motor de videojuegos con diferentes estados de ejecución.

11. EVIDENCIAS

Para demostrar el funcionamiento de la práctica se pueden agregar capturas de pantalla de los principales estados del programa:

Evidencia 1. Ejecución de `TitleScene`.



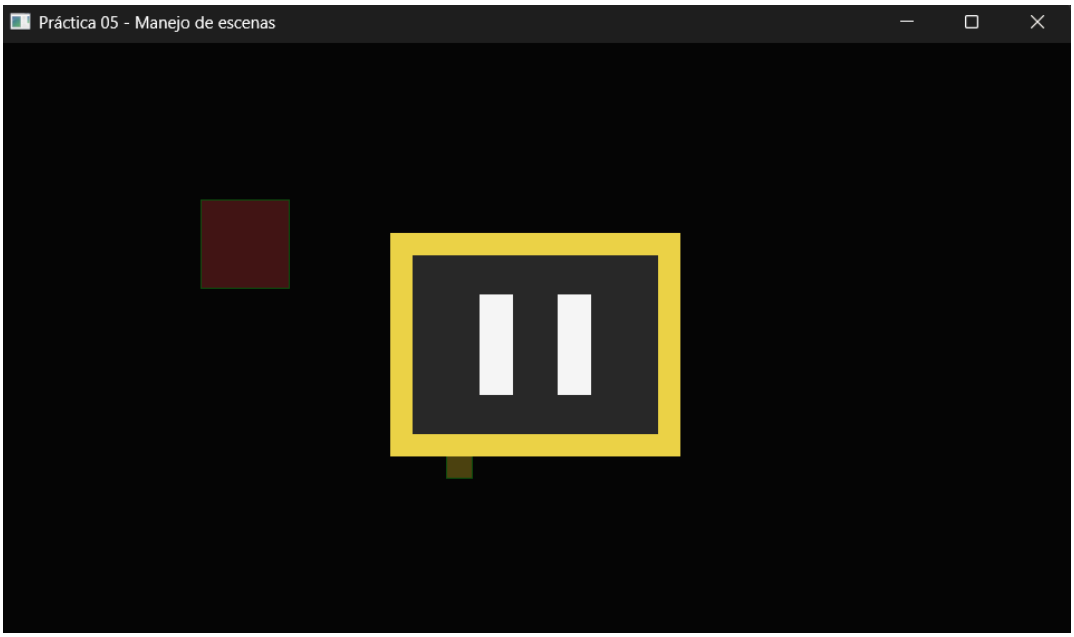
Evidencia 2. Ejecución de `GameScene` mostrando al jugador, la pelota y el obstáculo.



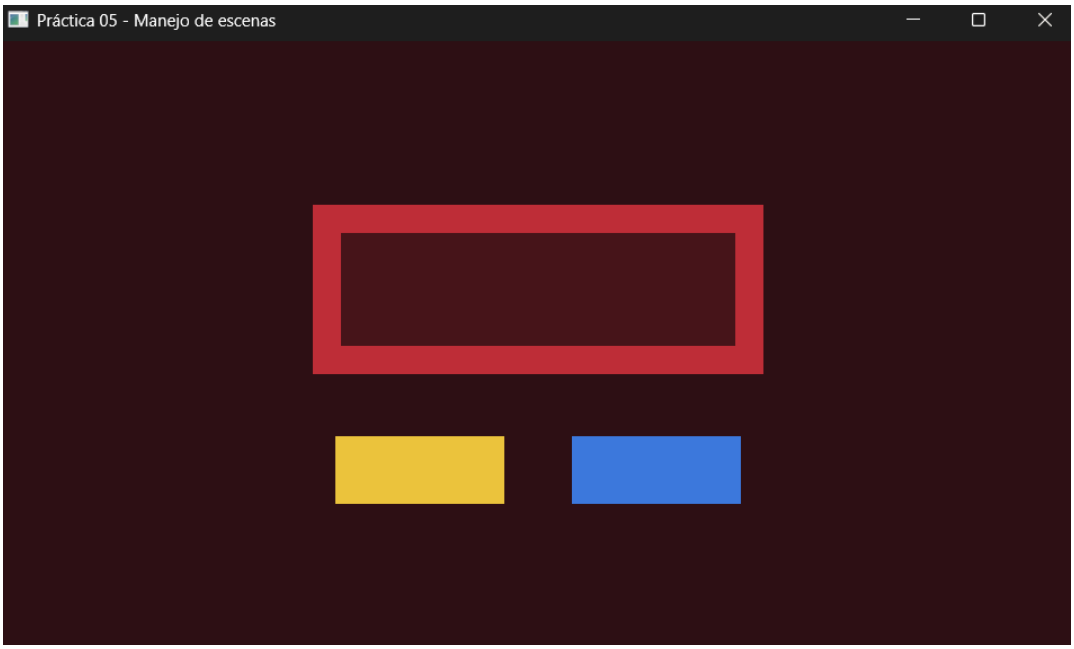
Evidencia 3. `GameScene` con el Debug Draw de colliders activado mediante F1.



Evidencia 4. Ejecución de `PauseScene` sobre la partida.



Evidencia 5. Ejecución de `GameOverScene`. (Este se ejecuta si presionas la tecla “G” del teclado)



Evidencia 6. Terminal mostrando la compilación correcta de practica05_motor.exe:

```
ellio@Piabonnie UCRT64 /c/motor de videojuegos C++/semana02/semana02/plantilla
$ ./build/practica05_motor.exe
Entrando a TitleScene
Saliendo de TitleScene
Entrando a GameScene
Collision: Ball con Player
Collision: Ball con Obstacle
Collision: Ball con Player
Debug Draw: ACTIVADO
Collision: Ball con Player
Collision: Ball con Player
Collision: Ball con Obstacle
Collision: Ball con Player
Collision: Ball con Obstacle
Entrando a PauseScene
Saliendo de PauseScene
Saliendo de GameScene
Entrando a GameOverScene
```

11.5. EVIDENCIA DE GITHUB

Repositorio:
https://github.com/Piabonnie-Dev/practica02-motor

Tag de entrega:
v0.5

Enlace directo al tag:
https://github.com/Piabonnie-Dev/practica02-motor/tree/v0.5

12. REFERENCIAS

Para el desarrollo de esta práctica se utilizó como base el código realizado durante las prácticas anteriores del motor, principalmente los sistemas de componentes, transformaciones, renderizado, movimiento y detección de colisiones desarrollados hasta la Práctica 04.

También se utilizó el material proporcionado para la **Práctica 05: Manejo de escenas**, principalmente para implementar el patrón de escenas, la pila de estados, las transiciones diferidas y el funcionamiento de `TitleScene`, `GameScene`, `PauseScene` y `GameOverScene`.

Como apoyo técnico se utilizó la documentación de SDL3 para las funciones relacionadas con eventos, renderizado, creación de ventanas y manejo de entradas.